

Formulários + PHP: o site ouve o usuário

Texto de apoio para falar sobre as Aulas 37 a 42 — Parte IV: GET e POST, validação, sanitização e o fluxo completo formulário → processamento → resultado.

AULAS 37 A 42 · PARTE IV · SEMANAS 13 E 14

Material de apoio da disciplina · baseado no Plano de Ensino e no Cronograma oficiais · todos os softwares citados são gratuitos.

TOM DE CONVERSA · AULAS 37 A 42

O textinho (para falar ou ler com a turma)

Abrindo o assunto

Até agora, nossos formulários eram bonitos e mudos: o usuário digitava e... não acontecia nada. Neste bloco, o site finalmente **ouve e responde**: os dados digitados viajam até o PHP, que recebe, confere, protege e devolve uma confirmação. É o coração de todo sistema Web.

Aulas 37 e 38 — GET e POST: duas formas de enviar

Com **method="get"**, os dados viajam **na própria URL** (?produto=notebook) — ótimos para buscas e filtros, péssimos para senhas (todo mundo vê). Com **method="post"**, viajam **escondidos no corpo da requisição** — o jeito certo para cadastros e logins. No PHP, a leitura é espelhada: **\$_GET** e **\$_POST**, dois arrays associativos automáticos cuja chave é o **name** de cada campo.

Aulas 39 e 40 — Validar e sanitizar: a dupla de segurança

Regra de ouro: **nunca confie no que o usuário envia** — o required do HTML pode ser contornado. Por isso o servidor confere com **isset()** (veio?) e **empty()** (veio preenchido?), coletando todos os erros de uma vez. E na hora de **exibir** qualquer dado, **htmlspecialchars()** transforma **<** e **>** em entidades inofensivas — a vacina contra o ataque **XSS**, em que alguém tenta enviar código no lugar de texto.

Aulas 41 e 42 — O fluxo completo, de ponta a ponta

Juntando tudo: formulário HTML com POST > PHP valida campo a campo > se houver erro, lista tudo com link "voltar" e para (exit) > se estiver tudo certo, sanitiza e exibe a **confirmação**. Esse fluxo de quatro etapas (preencher, enviar, processar, responder) é exatamente o que você vai repetir a vida inteira como programador Web — e é o núcleo do projeto: **cadastro.html + processar.php**.

Fechando a ideia

Um site que só exibe é um mural; um site que recebe, valida e responde é um **sistema**. GET consulta, POST cadastra; **isset/empty** conferem; **htmlspecialchars** protege. Com esse bloco, seu projeto final já tem espinha dorsal.

O ESSENCIAL DE CADA ETAPA**Em uma frase (para escrever no quadro)**

- **Aulas 37–38:** GET envia na URL (\$_GET, buscas); POST envia no corpo (\$_POST, cadastros e senhas).
- **Aulas 39–40:** isset/empty validam no servidor (required não basta); htmlspecialchars sanitiza a saída contra XSS.
- **Aulas 41–42:** fluxo completo: preencher > enviar > validar/processar > confirmar; projeto cadastro.html + processar.php.

TERMOS PARA DEIXAR NO QUADRO

method · action · GET · POST · \$_GET · \$_POST · name · requisição · corpo · URL · isset() · empty() · validação · servidor · sanitização · htmlspecialchars() · XSS · exit · confirmação · fluxo em 4 etapas

PERGUNTAS ORAIS PARA FECHAR O BLOCO**Para fixar conversando**

1. Quando usar GET e quando usar POST? Dê um exemplo de cada.
2. O que `isset()` e `empty()` verificam, respectivamente?
3. Por que não basta o `required` do HTML?
4. `htmlspecialchars()` protege contra qual ataque? Como?
5. Quais são as 4 etapas do fluxo de um formulário com PHP?

GABARITO RÁPIDO DO PROFESSOR

1) GET para buscas/filtros (dados na URL); POST para cadastros/logins (corpo da requisição). 2) `isset` = foi enviado; `empty` = está vazio. 3) Porque pode ser contornado (F12); a validação séria é no servidor. 4) XSS: converte `<` `>` `&` aspas em entidades, então o código vira texto. 5) Preencher > enviar > processar/validar > responder.